Kinematic Analysis Report

Velocity & Acceleration Kinematics Implementation and MuJoCo Validation for the Lite6 Manipulator

Robotics Project Milestone 3 - MCTR911 - Team 2

1. Executive Summary

This report outlines the analytical formulation, computational implementation, and empirical validation of the forward and inverse **velocity kinematics** and **acceleration kinematics** for the 6-DOF Lite6 robotic arm. The mathematical framework relies on Standard Denavit-Hartenberg (DH) parameterization. The implementation was rigorously validated via unit self-tests (analytic vs. finite-difference derivative comparisons) and cross-validated against MuJoCo physics simulation sensing routines.

2. Kinematic Structure & DH Parameters

The Lite6 manipulator features 6 revolute joints. The forward position kinematics computes link transformations using the standard DH convention:

$$^{i-1}T_i = R_z(\theta_i)\, T_z(d_i)\, T_x(a_i)\, R_x(\alpha_i)$$

To match the physical model defined in `lite6.xml`, specific joint angle offsets ($\theta_{offset2} = \theta_{offset3} = -\pi/2$) were applied. The exact parameters used in the implementation are listed below:

Joint (i)	θ_i Offset (rad)	d_i (m)	a_i (m)	α_i (rad)	Joint Limits [Min, Max] (rad)
1	0.0	0.2435 (L1)	0.0	-π/2	[-2π, 2π]
2	-π/2	0.0	0.2002 (L3)	π	[-2.618, 2.618]
3	-π/2	0.0	0.0870 (L5)	π/2	[-0.061, 5.236]
4	0.0	0.22761 (L6)	0.0	π/2	[-2π, 2π]
5	0.0	0.0 (L7)	0.0	-π/2	[-2.164, 2.164]
6	0.0	0.0625 (L8)	0.0	0.0	[-2π, 2π]

3. Velocity Kinematics Analysis

3.1 Geometric Jacobian Matrix

The end-effector twist vector is defined in the base frame as $V = [v^T, \omega^T]^T \in \mathbb{R}^6$, comprising linear velocity v and angular velocity ω. The geometric Jacobian $J(q) \in \mathbb{R}^{6\times6}$ maps joint velocities $\dot{q}$ to end-effector velocity:

$$V = J(q)\,\dot{q} \quad \Rightarrow \quad J_i(q) = [\,(z_{i-1} \times (p_e - p_{i-1}))^T\,,\, z_{i-1}{}^T\,]^T$$

where z_{i-1} is the unit rotation axis of joint i, p_{i-1} is the origin of frame i-1, and p_e is the end-effector position in the base frame.

3.2 Inverse Velocity Kinematics & Damped Least Squares (DLS)

Inverting joint velocity directly via $\dot{q} = J^{-1} V$ fails near kinematic singularities where $det(J) \to 0$. To maintain numerical stability across all configurations, a singularity-adaptive Damped Least Squares (DLS) solver is implemented via Singular Value Decomposition (SVD):

$$\dot{q} = V \cdot diag(\sigma_i / (\sigma_i^2 + \lambda^2)) \cdot U^T \cdot V_{twist}$$

The damping parameter λ^2 smoothly activates only when the minimum singular value σ_{min} drops below a predefined threshold $\varepsilon = 0.05$:

$$\lambda^2 = [\, 1 - (\sigma_{min} / \varepsilon)^2 \,] \cdot \lambda_{max}^{\ 2} \qquad \text{(where } \lambda_{max} = 0.05\text{)}$$

4. Acceleration Kinematics Analysis

4.1 Forward Acceleration Kinematics

Differentiating the velocity kinematic relationship $V = J(q)\ \dot{q}$ with respect to time yields the end-effector spatial acceleration $V = [a^T, \alpha^T]^T$:

$$V = J(q)\ \ddot{q} + J(q, \dot{q})\ \dot{q}$$

Here, $J(q, \dot{q})$ represents the exact time derivative of the Jacobian matrix. In `kinematics.py`, J is computed analytically column-by-column by tracking joint axis rate of change $\dot{z}_i = \omega_{i-1} \times z_i$ and joint origin velocities $\dot{p}_i = \sum_{k<i} (z_k \times (p_i - p_k))\ \dot{q}_k$:

$$J[:3, i] = \dot{z}_i \times (p_e - p_i) + z_i \times (v_e - \dot{p}_i), \qquad J[3:, i] = \dot{z}_i$$

4.2 Inverse Acceleration Kinematics

Given a desired spatial operational space acceleration V_{target}, the required joint accelerations $\ddot{q}$ are computed by isolating the acceleration term:

$$J(q)\ \ddot{q} = V_{target} - J(q, \dot{q})\ \dot{q} \quad \Rightarrow \quad \ddot{q} = J^{-1}(q)\ [\, V_{target} - J(q, \dot{q})\ \dot{q}\,]$$

In singular or ill-conditioned states, the DLS solver is applied to the right-hand-side vector $b = V_{target} - J\ \dot{q}$ to ensure well-behaved solutions.

5. Validation Methodology & Results

5.1 Standalone Unit Testing & Finite-Difference Verification

The module features a comprehensive self-test suite (`self_test()`) performing 200 randomized configuration evaluations to confirm mathematical consistency independent of external simulators:

- **FK Verification:** Standard DH FK matched direct MJCF link transformation chain.
- **Jacobian Derivative:** Analytic J compared against central finite difference $(J(q+h\ \dot{q}) - J(q-h\ \dot{q}))/(2h)$ with step size $h = 10^{-6}$.

- **Acceleration Forward Check:** Compared $\boldsymbol{J}\,\ddot{\boldsymbol{q}} + \dot{\boldsymbol{J}}\,\dot{\boldsymbol{q}}$ against numeric differentiation of spatial velocity along quadratic trajectories.
- **Round-Trip Checks:** Verified $\boldsymbol{IK(FK(\dot{q}))} = \dot{\boldsymbol{q}}$ and $\boldsymbol{IK(FK(\ddot{q}))} = \ddot{\boldsymbol{q}}$.

Test Item	Description	Worst-Case Error	Tolerance	Result
`fk_vs_xml`	DH Forward Kinematics vs. MJCF XML Tree	$< 5.0 \times 10^{-5}$ m	5.0×10^{-5}	PASS
`J_vs_fd`	Jacobian Velocity vs. Finite Difference FK	$< 1.0 \times 10^{-6}$ m/s	1.0×10^{-6}	PASS
`Jdot_vs_fd`	Analytic Jdot vs. Central Difference Jdot	$< 1.0 \times 10^{-6}$ s^{-1}	1.0×10^{-6}	PASS
`acc_vs_fd`	Forward Acceleration vs. Numeric Trajectory Diff	$< 1.0 \times 10^{-5}$ m/s^2	1.0×10^{-5}	PASS
`vel_roundtrip`	Inverse Velocity Kinematics Round-Trip	$< 1.0 \times 10^{-6}$ rad/s	1.0×10^{-6}	PASS
`acc_roundtrip`	Inverse Acceleration Kinematics Round-Trip	$< 1.0 \times 10^{-5}$ rad/s^2	1.0×10^{-5}	PASS
`ik_position`	Iterative Numerical Position IK Accuracy	$< 1.0 \times 10^{-5}$ m	1.0×10^{-5}	PASS

5.2 MuJoCo Physics Cross-Validation & ROS2 Real-Time Monitoring

In `compare_with_mujoco()` and `kinematics_monitor.py`, theoretical output is cross-validated against MuJoCo simulation state sensing (using `mj_objectVelocity`, `mj_objectAcceleration`, and central difference sensing):

> **Real-Time Monitor Design (`kinematics_monitor.py`)**
>
> The monitor subscribes to ROS2 joint state updates (`/joint_state`). It maintains a rolling buffer over a 10ms window, estimates joint accelerations $\ddot{\boldsymbol{q}}$ via linear least-squares regression over published joint velocities, and evaluates equation vs. sensed twist/acceleration at 20 Hz.

6. Conclusion

The velocity and acceleration kinematic equations developed for the Lite6 robot arm demonstrate exact convergence with analytical derivatives and simulation measurements. The incorporation of analytical $\boldsymbol{J}$ enables highly accurate operational-space acceleration control, while the SVD-based adaptive DLS guarantees robust performance across singular boundaries.